RA weekly Report

Hariom Mewada
24M2137

March 2, 2026

Contents

1. Octobor 14, 2024 - Octobor 21, 2024

1.1 Understood the intenton UI code

1. understood various modules of the intentor app like how **dashboard**, **Challenges** and **Daily usage limit** works .
2. Understood **Count of locks and usage count Notification**, **Login**,**alertlaunch** subsection of the App.
3. Went through code to understand their working.

1.2 Setting up Android Studio and Intentor on local machine

1. Downloaded **JDK** ,**Android Studio**,**Github**.
2. Cloned intentor repo and checkout to to **Master** branch to fetch latest code.
3. Setup environment for packages installation and **JDK** for running Android Studio.
4. Understood various terminology and concepts related to **Android Development**

1.3 Development of Digital Detox Library

1. Started implementation of library with various usefull features like **Session**, **Scheduled Session** ,**Pause** ,**InterSessionGap**, **Challenges** which can further extend the functionality of intentor App very smoothly.

1.4 Future Goals

1. **Add Customizable Challenge Types:** Expand the challenge types for users to include mental, physical, and cognitive tasks to further discourage excessive social media usage.
2. **Enhance Challenge Verification:** Implement stronger challenge verification mechanisms, like integrating quizzes, puzzles, or tasks requiring user interaction (e.g., image or voice recognition).
3. **Usage Insights & Analytics:** Build a system to track app usage patterns and provide personalized feedback to users based on their activity levels, including metrics like total screen time, app usage frequency, and longest continuous sessions.
4. **Notifications and Reminders:** Add features that remind users to complete challenges or display motivational messages when their usage time exceeds the set limit.
5. **User Profiles and Data Syncing:** Store user preferences and challenge progress in a profile system that supports cloud syncing, so users can manage their data across devices.

1.4.1 Research and Testing

1. **Behavioral Research:** Study behavioral trends around social media addiction to refine the library and create more effective mechanisms for user control.
2. **Usability Testing:** Conduct usability studies with a group of users to gather feedback on the overall user experience.

2. Plans for RA work

2.1 Analyze and Optimize Existing Features

- **Review User Feedback**: Gather and analyze feedback from current users to identify pain points or missing features.
- **Bug Fixing and Optimization**: Identify and resolve existing bugs or performance issues to improve the user experience.
- **UI/UX Enhancements**: Update the interface to make it more user-friendly and visually appealing.
- **Explore similar product in market**: Explore and analyze feature offered by other similar applications in market

2.2 Enhance Functionality

- **Customizable Challenges**: Allow users to select preferred friction challenges or adjust their difficulty levels.
- **App Usage Analytics**: Implement a dashboard showing detailed usage statistics (e.g., time spent on apps, frequency of unlocking challenges).
- **Notification Scheduling**: Add a feature to schedule reminders or restrict app access during specific times.

2.3 Strengthen App Monitoring

- **Foreground App Detection**: Improve the accuracy of identifying apps in the foreground and detecting patterns.
- **Battery Optimization**: Optimize the monitoring service to minimize battery consumption.

2.4 Introduce New Features

- **Gamification**: Reward users for reducing their social media usage or successfully solving challenges.

2.5 Technical Improvements

- **Compatibility**: Ensure compatibility with the latest Android version.
- **Security Enhancements**: Conduct a security audit to protect sensitive user data and implement best practices.
- **Library and Dependency Updates**: Update outdated libraries and dependencies for better performance and security.

2.6 Testing and Deployment

- **Comprehensive Testing**: Perform rigorous testing, including unit, integration, and stress tests.
- **Beta Testing**: Launch a beta program to collect feedback before a public release.

2.7 Documentation

- **Code Documentation**: Ensure the codebase is well-documented for easier maintenance.

3. Dec 10 2024 - Dec 13 2024

3.1 Testing intentor App on Android version 13

1. Upgraded android studio version to run emulator
2. set up android 13 device in emulator.
3. upgraded gradel and and gradle puglin version to resolve version mismatch error
4. successfully tested pop up opening feature .
5. pop up was opening fine in the emulator
6. tested it on the latest intentor code.

3.2 Understood implemetation Details of Intentor through omkar's work

3.2.1 Logging and Data Collection

1. undestood how logs are created in **(app logs.txt)** file by intentor app and how they are stored in **Internal Storage**,**External Storage** ,**Reading and Writting Files** and every 3 hour api is called to update log to the server. we can utilize these logs for further analytics.
2. will test this feature end-to-end once our server is up.

3.3 Understanding various types of service available in Android to use in Intentor

3.3.1 Introduction

In Android, **services** are components used to perform long-running operations in the background without a user interface. These services continue running even when the user switches to another application. Below is a breakdown of the types of services in Android:

3.3.2 Foreground Service

- **Definition:** Runs in the foreground and is highly visible to the user. It typically shows a persistent notification in the status bar.
- **Use Case:**
  * This service we are using for starting another background service which will open popup
  * Playing music in the background.
  * Tracking user location for navigation.
  * File downloads or uploads.
- **Key Features:**
  * Users are aware of the service as it is running.
  * Must display a notification when started.

3.3.3 Background Service

- **Definition:** Runs in the background without a user interface. It performs tasks that do not require user interaction.
- **Use Case:**
  * Syncing data with a server.
  * Performing data backup.
- **Key Features:**
  * Runs in the background for a limited time after being started.
  * In Android 8.0 (Oreo) and above, background services are restricted for apps running in the background. Developers are encouraged to use **WorkManager** or **JobScheduler** for tasks instead.

3.4 Bound Service

- **Definition:** Provides a client-server interface that allows components (such as activities) to interact with the service.
- **Use Case:**

    - Apps where multiple components need access to the same service instance (e.g., a music player app allowing controls from different parts of the app).
- **Key Features:**
    - Bound to an activity or another component via `bindService()`.
    - Terminates automatically when no component is bound to it.

3.5 Intent Service (Deprecated)

- **Definition:** A subclass of `Service` designed to handle asynchronous requests on a worker thread.
- **Use Case:**
    - Tasks such as downloading or processing files.
- **Key Features:**
    - Handles one intent at a time in a background thread.
    - Stops itself when the task is complete.
    - Recommended to use **WorkManager** for these tasks after deprecation.

3.6 Started Service

- **Definition:** A service that is explicitly started using `startService()`.
- **Use Case:**
    - Long-running tasks such as playing music, downloading files, or performing heavy computations.
- **Key Features:**
    - Continues running until stopped explicitly with `stopSelf()` or `stopService()`.

3.7 Alternatives to Traditional Services

Due to battery and performance concerns, Android introduced newer APIs:

- **JobScheduler:** For scheduling background tasks based on conditions (e.g., charging, network availability).
- **WorkManager:** For deferrable and guaranteed background work (recommended for tasks like syncing data).
- **Foreground Service with Notification Channel:** For user-visible tasks in newer Android versions.

Service Type	Persistency	User Interaction	Examples
Foreground Service	Persistent	Yes	Music player, location trackin
Background Service	Limited in scope	No	Syncing data, backups
Bound Service	Depends on client	Yes (via binding)	Shared resource, media player
Intent Service (Deprecated)	Temporary	No	File uploads, processing data
Started Service	Until stopped	No	Long-running tasks (e.g., dow

Table 1: Summary of Android Services

4. Dec 15 2024 - Dec 19 2024

4.1 Breaking Down User Feedback Testing in More Concrete Steps

4.1.1 Defining The Goals

1. We will clearly outling what information we want to gather from the user.
2. Are there specific pain points users face?
3. Are there features users would like to see added?
4. is the really helping them in controlling social media usage?
5. Are there usability or performance issues?

4.1.2 Choose a Feedback Collection Method

1. **Online Surveys:** We will create Google form for online servery or create a whatsapp group for Social Media Polls.
2. **Types of Questions:** We can Include a mix of multiple-choice questions, rating scales, and open-ended questions
3. **Example Questions:**
   (a) What do you like most about the app?
   (b) What challenges have you faced while using the app?
   (c) What new features would you find valuable?
   (d) What improvement can be done in currently implemented features.
   (e) what they like about UI?
   (f) any feedback on currenly UI?

4.1.3 Collect and Analyze Feedback

1. **Compile Data:**Export survey responses into tools like Excel, Google Sheets
2. **Categorize Responses:** Categorize feedback into common themes or topics (e.g., feature requests, usability issues).
3. I can do parallel testing of features at my end as well
4. After user feedback we can make changes in our app those seems more intuitive or relatable.

4.2 Setting up backend server for intentor on local and connecting it to application

1. Downloaded Docker and up the backend server in docker container on my local.
2. Run the application on my local while connecting with the local backend server.

4.3 UnderStanding More about Android

I have explored various concepts and technologies essential for Android development. The key concepts I have learned include:

4.3.1 Android Fundamentals

1. **Activities and Tasks:** Explored the lifecycle of an Activity, the role of tasks, and how multiple activities are managed within the task stack.
2. **Android Main Thread and UI Thread:** Understood the importance of the main thread in managing the UI and the role of worker threads to prevent ANR (Application Not Responding) errors.
3. **Parcelable:** Learned to pass objects efficiently between activities or fragments using the Parcelable interface.
4. **AsyncTask:** Studied how to perform background tasks with AsyncTask, including pre- and post-execution callbacks.

4.3.2 Essential Android Components

- **Services:** Explored how to create background services to perform long-running tasks, including foreground services for ongoing user interaction.
- **Broadcast Receivers:** Learned to listen to and act upon system-wide or app-specific broadcast messages.
- **Content Providers:** Understood the role of content providers for data sharing between applications and accessing external data sources.
- **Android Permissions:** Explored permission handling, including runtime permissions for sensitive data access.

4.4 Data Storage and Management

1. **SQLite Database:** Learned SQLite for local data storage and learned CRUD (Create, Read, Update, Delete) operations in Android.

4.5 Market Research and Observations

I explored similar products available in the market. Key features of these products include:

1. Add Application guide and terms and condition for app usage then permission activity when the user open the application for the first time
2. User-friendly and intuitive UI designs.
3. Ability to block specific parts of applications, such as feeds or messaging.
4. Integration of multiple devices for aggregated data analysis and synchronized control.
5. Restriction settings on an hourly or daily basis, allowing for flexible usage management.
6. Blocking access to certain websites to enhance focus and productivity.
7. Parental control features, including app activity monitoring and detailed usage reports.
8. notifications and alerts for reaching usage limits.

5. Dec 30 2024 - Jan 03 2024

Mention time spent in hours

5.1 Implementation Of Onboarding Screen

1. tried implementing on-boarding screen using some AppIntro Library.
2. now implemented on boarding screen using viewpager2
3. pending works are like adding shared preference for showing onboarding screen for one time only and editing background picture .
4. implemented shared preference to ensure the activity will open only once.
5. next goal is customize the screen and background images .
6. Also need to add permission slides with proper explanation.
7. assisting in user study.

5.2 Plan for making Introductory video for user study

1. **Introduction (0:00 - 0:15):** Briefly introduce about the speaker and the app.
2. **The Problem (0:15 - 0:30):** Discuss the challenges of excessive social media use.
3. **App Features and Demo (0:30 - 1:30):** Highlight the main features with visuals or screen recordings.

4. **Benefits (1:30 - 1:45):** Explain the value users will get from the app.
5. **Call to Action for User Testing (1:45 - 2:00):**Invite users to test the app.Provide clear instruction for participation.

6. Jan 06 2025 - Jan 10 2025

6.1 Design the On Boarding Screen

1. i have searched for some pictures but i was not able to find perfect background pic that is more related to app.
2. I am exploring on UI part of android .
3. Completed vedio creation for user study but i need to edit it . I need to find out some software which are free and fullfill our requirement. I have tried on mod apk but i was not able to edit it perfectly.

6.2 Audio Narration Script for Introductory Vedio:

6.2.1 Introduction

- "Hi, meet Intentor, a social media usage control app designed to help you stay focused and manage your screen time. We know how easy it is to get distracted by social media. That's why we created this app to give you tools to build healthier habits."

6.2.2 Feature Demonstration

- "Whenever you open a social media app, Intentor steps in with an overlay popup. It's your personal reminder to pause and reflect. Here, you can choose to continue to social media app for 5 to 20 minutes and again pop up will appears or you can quit the social media app right at the moment or you can mute alters for next 2 hours for this perticulare app if you have some urgernt work aligned with that app or you can take a challenge like 'Digital Fasting' or 'Dieting challenge' or 'App Access Deiting Chellenge' to stay on track."
- (Show selecting a challenge) "For example, with the 'Fasting Challenge,' you commit to not using social media for the two few hours.with Dieiting challenge you can limit social media usage to one hour per day and with App Access Dieting Challenge you can limit Accessing the to 20 times a day . This builds discipline and reduces unnecessary scrolling."

6.2.3 Benefit

- "Intentor makes it easier to break free from distractions, focus on what truly matters, and develop mindful social media habits."

6.2.4 Call-to-Action

- "Ready to take control of your screen time? Download Intentor today and start your journey to a balanced digital life."

7. Jan 12 2025- Jan 17 2025

BR: Mention number of hours. Correct year.

1. Called and texted around 43 people to onboard for user study.
2. Around 20 people started using the app.
3. Had a meeting with interns on Sunday morning but rescheduled it because they does not had github access.
4. Got feedback from one student for ui: UI suggestions: Time Bar should be around navigation area. Use dark and light themes for colors. Gradient for premium users. Floating notification for time device is on since unlocked.
5. got one more feedback from 2 students that: **Instander** is also on 10 mins limit. we can explore this app as well.
6. one user reported that he can't customize challenge duration.
7. one user reported that stat's are not very clear,it's confusing for new users.
8. timer floating should not be on the centre instead it should be on screen's corner.
9. mute setting should be customizable from app not from popup.
10. functionality of unmute should be added.
11. challenge should impose hard restriction . user should not have the freedom to quit the challenge via popup ,instead we can dirctly show that you can open app after the remaining time.
12. one user suggested that instagram reels or youtube shorts are never ending ,so we should derive some solution on the count of reels watched or shorts watched and restrict the user from further using the SM app after some customizable amount of time.

8. Jan 19 2025 - Feb 4 2025

8.1 Time spent

1. 4 hours in reading , testing the code writing and drawing state machine diagram and 1 hour in meeting .

8.2 List of tasks:

1. Went through the theory of creating machine state diagram.
2. Went through how to set debug = False and change settings.py to load the static files and run the command python manage.py collectstatic
3. Created state machine diagram in my notebook.
4. Done meeting with one of the interns to resolve their emulator issue.

8.3 Some Highlights from implemenation of popup

1. Monitor forground service is started by a Broadcast reciever named UnlockReciever which is static reciever, it is set to recieve broadcast when the device is unlocked or device is boot up .
2. Monitor foreground service starts a thread which check in every half second for the social media app to come foreground.
3. Checks performed by Monitor:
   (a) Collect all the usageStats of last two second from UsageStateManager as a list of usageStats.
   (b) Iterate over the list of usageStats and for every usageStats check if the package name is of our interest i.e. social media app and its time stamp is should be in last two mintues .
   (c) Equation that needs to be satisfied:

$$T_{\text{lastused}} \geq T_{\text{current}} - 1000$$

4. Based on the above checks start the overlay service which will show popup if it is not already upeared.
5. Phone usage calculation to show on top of popup: Query all the usage Events of the day and calculate the time difference btw background and foreground events need to see if the events returned by android are sorted based on the time.
6. Will add details of setting up popup if needed.

8.4 Overview of some similar product explored

8.4.1 Minimalistic phone

1. Beautiful on-boarding slides explaining each and every feature ,total 11 slides.
2. Good looking Ui with mostly in black background.
3. features:
   (a) Icon free simple interface of mobile phone.
   (b) App blocking

(c) In-App time reminder.

(d) mindfull launch delay of 15 seconds.

(e) Organize apps in folder to create favorites or some other groups of app.

(f) Hide apps

(g) Filter notifications

(h) Monochrome mode: Enjoy a more peaceful and less visually stimulating experience by rendering selected apps in black and white. Especially effective in image-focused apps, social media , and games.

8.4.2 AppBlock

1. Have option to show daily weekly usage.
2. Have option to see launch count ,daily usage time, unlock count .
3. Block notifications
4. Devide the day in study time, focus,Add app limits,Time for my self, Time at home, time at work and add the app to block list of particular category.
5. Gave options to strick mode with various customization like hard and soft canclable alert.
6. Block some keywords,also block websites like porn websites can also block some browsers,

9. Feb 5 2025 - Feb 12 2025

9.1 Time spent

1. 4 hours in reading about tate machine diagram, drawing state machine diagram and searching in code.

9.2 Current state machine diagram

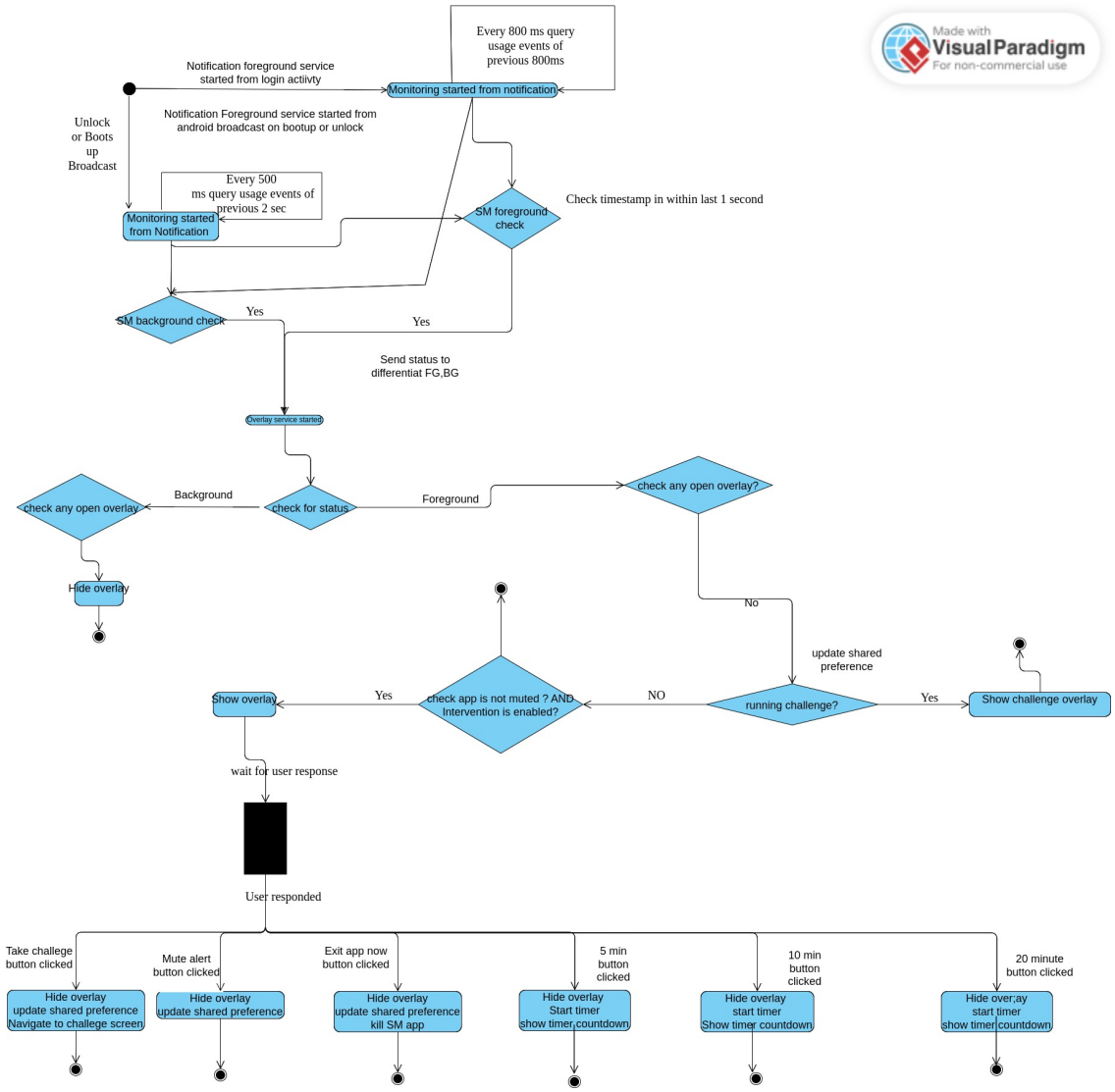


Figure 1: Current state machine diagram

9.3 Correct state machine diagram

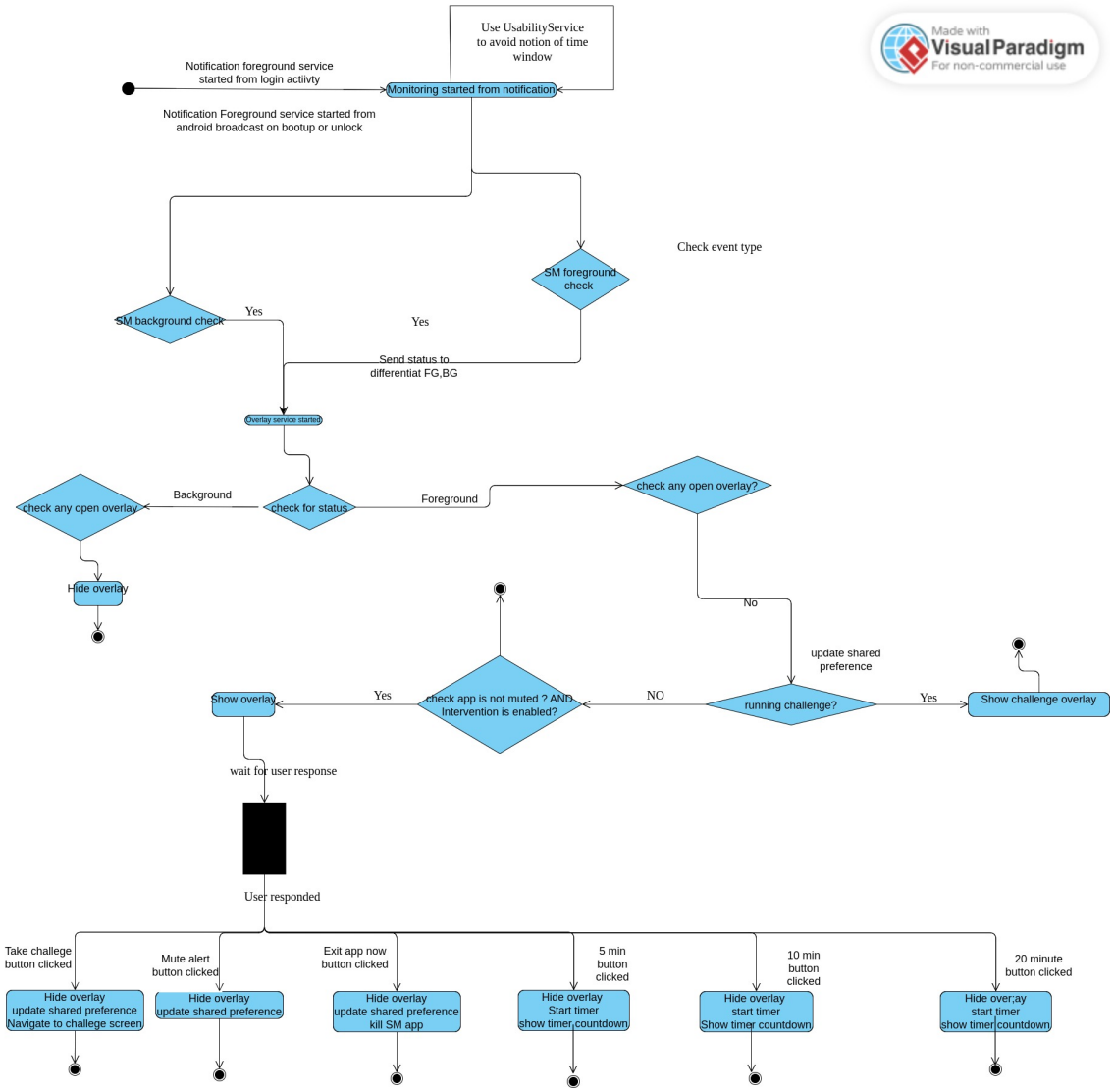


Figure 2: Correct state machine diagram

9.4 Reason to Use AccessibilityService Over Usage States and Usage Events

9.4.1 Real-time App Monitoring

1. **AccessibilityService** provides immediate callbacks when the foreground app changes.
2. **UsageStatsManager** relies on batch updates, leading to delays in detecting app usage.

9.4.2 More Detailed App Interaction Tracking

1. **AccessibilityService** can detect UI elements, button clicks, and user interactions inside apps.

2. **UsageStatsManager** only provides information on when an app was opened or closed.

9.4.3 App Blocking and Overlay Capabilities

1. **AccessibilityService** can detect app launches in real-time and immediately trigger an overlay or block access.
2. **UsageStatsManager** does not provide direct control over active apps.

9.4.4 Compatibility with Newer Android Versions

1. **UsageStatsManager** requires the `PACKAGE_USAGE_STATS` permission, which the user must manually enable.
2. **AccessibilityService** does not require manual user intervention once granted.

9.4.5 Detecting System-Wide Events

1. **AccessibilityService** can detect events such as app switches, keyboard inputs, notifications, and UI changes.
2. **UsageStatsManager** is limited to tracking app launch and exit times.

9.4.6 Background Execution

1. **AccessibilityService** runs continuously in the background without polling.
2. **UsageStatsManager** requires periodic querying to get app usage data.

9.4.7 When to Use UsageStatsManager Instead?

1. Helpful if wee **historical app usage data** (e.g., daily/weekly reports).
2. Helpful if our goal is to analyze past usage trends rather than **real-time monitoring and control**.

9.4.8 Conclusion for Social Media Usage Control App

1. Since real-time detection and overlay display are required, **AccessibilityService** is the better choice.

10. Feb 13, 2025 - Feb 17, 2025

10.1 Time spend

1. $6 + 4$ hours in adding accessiblity service, adjusting old implementation with current one for testing and testing it multiple time with smaller code changes for debugging and making new state diagram
2. Facing two issues which creating inconsitency in popup.

(a) when we click on exit app now when popup is on user's screen, another foreground event happens which redisplay the popup.

(b) when we inter chat of type something, foreground event for that input keyboard happens and it removes the timer and when we come out of typing, foreground event for social media happens which display the popup again. It is not suppose to display this time.

3. To implement usability service we also need usablility permission.

10.2 Key take away from meeting

(a) Fix the diagram.

(b) Fix the popup.

(c) create a google form after mid sem to collect information about system1 and system2 thinking. Do compaiging, install posters or QR code on hostel's notice board.

10.3 Revised state machine diagram

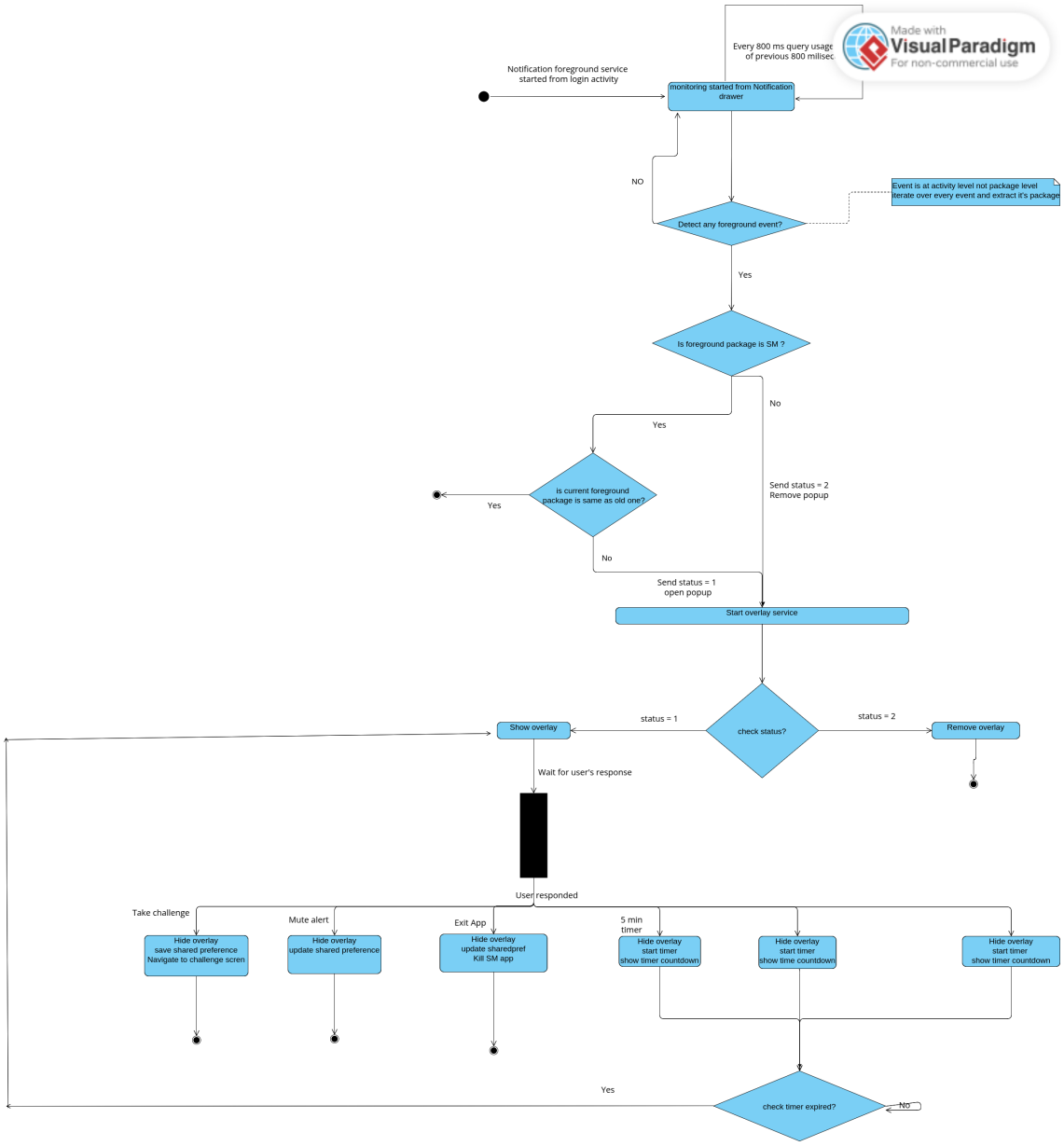


Figure 3: Revised state machine diagram

10.4 Revised imlementation

(a) Made overlay a foreground service so again and again start service will not happen .

(b) ignored background event fully, utilizing foreground event only to remove unneccessary edge cases.

(c) Reduced time of query to 500 ml from 800 ml second so time window is very small so there are less chances of missing any foreground event.

(d) Made code changes to use the notion of previos package to avoid to the inter app foreground background activity events.

(e) Impl emented and tested newly added changes.

11. Mar 2, 2025 - Mar 5, 2025

11.1 Time spent

(a) 5 hours in debugging,making code changes and testing.

11.2 summary of changes made

(a) Made changes in to query for last 1 second and this query will be made after every (500ms sleep + some miliseconds of work) miliseconds.

(b) Removed move to background events only considering foreground events.

(c) corrected unlock reciever now it can recieve a broadcast on unlock and restart the notifcation service if it was dead on every phone unlock.

(d) moved the timer pop to on top right corner of screen.

(e) Done some code refoctoring, we can refactor more if you want we can remove overlay service and open window manager directly from only single service.

12. Mar 19, 2025 - Mar 25, 2025

12.1 Time spent

1. 4 hours in implementing and testing the fix.

12.2 List of task done

(a) made 2 pushed to github related to popup fix.

(b) Fixed the popup not removing, timer not reseting when two consecutive social media is opened, app is getting killed from background, tested app on 3 devices for checking the order of logs of foreground and background.

(c) Reseting the timer when screen goes off.

(d) fixed the position of the timer on the top right corner of the screen.

13. May 22 2025 - May 26 2025

13.1 Time spent

1. 6 hours.

13.2 Summary of work done

1. Went through the scripts for generating aggregated stats : github link
2. **Stats and graphs covered so far in the script**
   (a) Calculate plot weekly average time usage per app.
   (b) Calculate plot weekly access count per app.
   (c) Calculate and plot the aggregated graph for access count and usage time in one go and write the generated stats in CSV file as well.
   (d) Generate CSV file for weekly usage time for every app.
   (e) Generate CSV file for weekly access count for every app.
   (f) Generate aggregated CSV file for weekly access count + weekly usage data (for all the weeks in single CSV file) for every app.
3. Setup and ran the backend application on my local.

14. May 27 2025 - June 02 2025

Number of hours: 6

14.1 TO DO for RA work

1. Utilize Accessibility service provided by android to extract more events from android which can be helpful in making interventions related how much the user scrolled and how many time time he had done pull-to-refresh etc.
2. Utilize Firebase database to get information about user's session on Instagram, YouTube etc.
3. Add script for stats generation for buttons click - tell exactly which graph.
4. `Graph of No of time the App opened vs No of time exit app now clicked vs No of time I am working or No of time i am bored Button is clicked.(Daily, Weekly, hourly, Any time etc.)`
5. `Similar graphs and stats can be generated for every Button in the server side.`
6. Add more stats and graphs in the existing scripts - tell exactly which graphs
7. Optimize the api for stats fetching as no of the users and data is very high on the server - which API? What is the current performance? `(Currently We are not using API only django based url endpoint but this feature is not working on server for fetching stats for all users in one go as amount of the data is very large.)`
8. Upgrade the application to latest Java version.
9. Make the code design good and Adaptive for future use.

10. As more user study will be going to happen in feature backend can be converted to Django Rest framework instead of just Django to make the application more modular and use REST API feature of Django framework.

11. Make the Intentor application compatible to work with newer android version (Android 15 +). Longer running services stopped working from Android 15+ which will impact our continous running notification drawer.

12. Make Intentor App to ask for setup first during installation (On boarding screens with guidance about onboard the application and ask permission on installation.).

13. Make user profile (will utilize this profile for showing customize interventions or notifications or ranks etc.)

14. Restructure the code to fit in common Design pattern to incorporate future changes in the project smoothly.

15. Currently server is running on http, will shift it to https for enhanced security .

15. June 03 2025 - June 09 2025

15.1 Investigating Working of Alarm in Android

Number of hours: 6

1. Alarm Manager API is responsible for alarm functionality in Android.

2. User usually know that there is background work going on.

3. With the help of Alarm Manager we can execute any peace of code or longer running task as well.

4. Calendar and Alarm app trigger specific reminder at specific time this reminder is time based and in Intentor reminder is event based i.e. when app opens up.

5. Work Manager is also an option but it's main purpose is for background task that need to be executed reliably and this task just take a while like synchronization or file upload, also user is not aware

6. Work Manager has limitation of minimum recurring period to be at least 15 min.

7. Alarm Manager is system service means it is coming from operating system, to use it, we can simply get reference to this using : *context.getSymtemService(AlarmManager.class)*

8. Bunch of API provided by Alarm Manager like setExact(to set exact alarm) or setInExact(we provide an interval in which our alarm should trigger) etc. API documentation

9. If our alarm receiver called Context.startService(), it is possible that the phone will sleep before the requested service is launched. To prevent this, our BroadcastReceiver and Service will need to implement a separate wake lock policy to ensure that the phone continues running until the service becomes available.

10. The Alarm Manager is intended for cases where we want to have our application code run at a specific time, even if our application is not currently running. For normal timing operations (ticks, timeouts, etc) it is easier and much more efficient to use Handler.

11. We can schedule an alarm on midnight and afternoon or every hour whose responsibility will be to start the foreground service which was killed by user or battery optimizer.

15.2 Following things are needed to make the services running for 24 * 7 hours

The user turns off battery optimizations for your app.

1. After the device reboots and receives the ACTION_BOOT_COMPLETED, ACTION_LOCKED_BOOT_COMPLETED, or ACTION_MY_PACKAGE_REPLACED intent action in a broadcast receiver.

2. The user turns off battery optimizations for our app.

3. Alarm Manager should periodically trigger an alarm which will start our service if it stopped.

4. Other foreground service type have restriction on time duration we can use FOREGROUND_SERVICE_TYPE_SPECIAL_USE to prevent our service from getting timed out in higher android version.

15.3 Current issue in Intentor

1. In my phone Intentor is working 24 * 7 hours as it has permission to ignore battery optimization.

2. Also intentor reciever broadcast on bootup.

3. Most probable issue of stopping the service is either the user killed the service from notification bar or system stopped the service as it does not have battery optimization permission.

15.4 Reason of Stopping Intentor App in Your Phone

1. Potential issue is missing *Ignore Battery optimization* permission in your phone because of which background service is getting killed by android operating system.

2. I am using Intentor in my phone (Moto G Android 14) since 4 days with *Ignore battery optimization permission* enabled, it is opening stable pop up (i.e. pop up goes off when we exit social media and pop up only appears for social media apps)

3. I tried without battery optimization permission, app gets stopped after some time.

15.5 Using Alarm Manager to restart the service after every hour or some custom interval

1. We can utilize Alarm Manager to trigger alarm after some hour which will restart the service.

15.6 Insights from Unplug Application

1. Notification gets triggered when service gets stopped or device boot-up .
2. When we click on the notification it will launch an activity, and from that activity intent is sent to the foreground service to start.

15.7 Meetings Key Decisions:

1. Ask permission to disable "Pause app activity if unused.".
2. Constant should be defined in different files and should be referenced whenever needed.
3. Usage Events Query time window should start from last time when the API was called.

16. June 10 2025 - June 16 2025

Number of hours : 6 + 4

16.1 List of Tasks Completed :

1. Created a new class file for all the constants and referencing them from that file.
2. Changed usage events query start time to ending time of last query.
3. In my phone "Pause app activity if unused" is enabled for Intentor but still it is running 24*7 hours.
4. BR: check in app (1) is battery optimization turned off, (2) is "pause app activity if unused" is turned off

16.2 Detail of the Implementation

1. BR: query starting from previous query time
2. We are only interested in latest foreground event.
3. If latest foreground event is from SM we are showing popup else we are hiding the popup .
4. Background Thread will sleep for 500 milliseconds after every iteration of work.

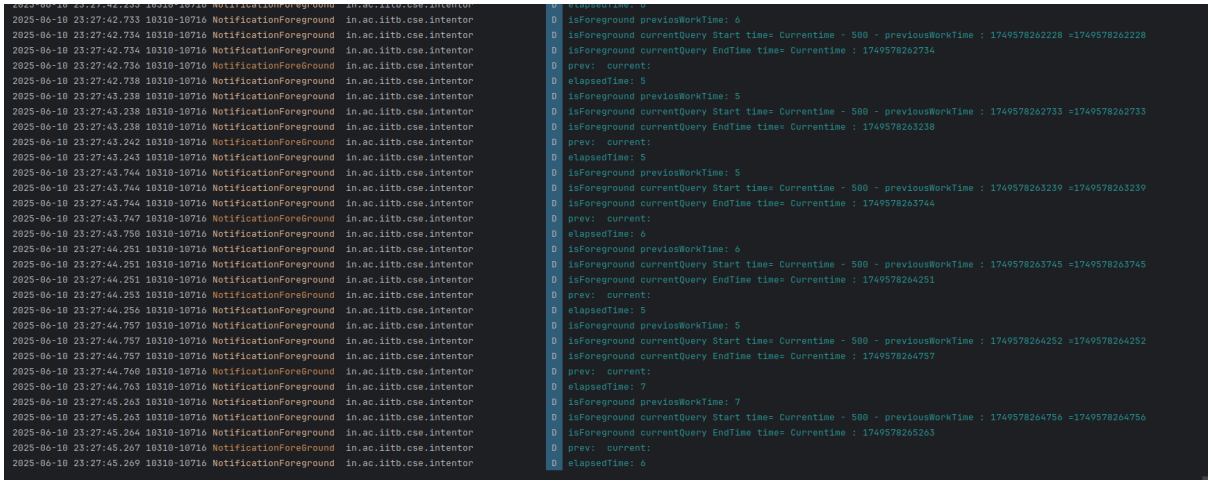


Figure 4: ss of the logs printed on terminal

5. *ElapsedTime* is the variable which stores time spent by the thread in previous execution.
6. Query start time of current execution is: *Currentime-sleeptime -ElapsedTime*
7. Query end time of current execution is *currentTime*.
8. After every iteration of work *ElapsedTime* gets updated and used in next iteration to find the start time of the query window.
9. Tested the changes on my phone.

16.3 Fix Query Starting Time

1. Changed Query start time to ending time of previous query.
2. Tested the changes on my device.

16.4 Battery Optimization permission

1. We have Already implemented asking run time permission to turnoff battery optimization for Intentor.

16.5 Runtime Permission of Disable Pause app activity if unused

1. Read about App Hibernation, but it happens when the app is not used for at least a month.
2. In Android there is no such run time permission: Available run time permission in Android
3. We can add an tutorial screen which will explain the user to grant this permission on run time.

4. Read about best practice to ask run time permission in Android, is to educate the user about why the permission is needed with the help of the animation,text, images which shows how to grant the specific permission. link

16.6 Insights from Stay Free App, Your Hour and Unplug Application

1. When Stay Free App has accessibility permission, "Pause App Activity if unused" is disabled and not clickable.

2. Your Hour app have battery optimization permission set to *Optimized*, not *Unrestricted* therefore it also get killed some time.

3. Stay Free don't run in background but still able to open pop .

4. Unplug runs in background and have battery optimization permission set to *Un-Restricted*, and does not have accessibility permission but here also , "Pause App Activity if unused" is disabled and not clickable.

17. June 17 2025 - June 23 2025

No. of hours: 6 + 6

17.1 List of task completed

1. Created on-boarding, permission and splash screen (total 5 screens)for Intentor app.

2. Added the flow to ask all the permission when the user visited the first time.

18. June 30 2025 - July 06 2025

No. of hours: 6

18.1 List of task completed

1. Resolved the flow of permission screens and add changes for asking permission if it's not granted already.

2. Resolved the bug from Notification foreground service. earlier the thread (used to track foreground apps) was running every milliseconds so it's behavior was inconsistent added Thread handler which automatically maintains thread pool for us we does not to worry about creating, stopping and rerunning the thread.

3. Refactored the code removed the unwanted comments and added constants used throughout the project in separate constants file.

19. July 07 2025 - July 13 2025

No of hours: 4

1. Added debug logs for foreground service in the log file.
2. tested logs are printing every 500 milliseconds.
3. Intentor already had two log file.
4. First log file server as primary log file and it is uploaded to the server every 3 hour by scheduling background task.
5. Second log file is overall log file and first log file is appeneded and to overall log file after it is uploaded to the server and reinitialized as empty file.
6. Cons: overall log file may become very large for longer course of duration we can create every day log file and upload it to the server and reinitialize the file.

20. July 13 2025 - July 19 2025

No. of hours: 20

20.1 List of Task Completed

1. Fixed dashboard to show stats of only installed apps.
2. Changed dashboard and challenges position on Bottom Navigation bar on dashboard.
3. pushed the changes to github
4. Added color coding in notification for usage time and unlock count.
5. Fixed bug that was triggering multiple notification alert.

21. July 21 2025 - July 27 2025

No. of hours: 21

1. Added settings to show color coding.
2. Made to the notification to visible on Locked screen.
3. Added boundaries for GREEN-YELLO and YELLO-RED zone in the graph.
4. Added sanity checks for input for usage time and unlock count threshold for number only
5. Added sanity checks for orange threshold must be less than red threshold.
6. Added about us page

7. Added checks for maximum limit of red zone for usage time is 360 minutes and orange zone is 180 minutes
8. Added checks for maximum limit for red zone for unlock count is 100 unlocks and and for orange zone is 50 unlocks.
9. Added lock screen overlay.
10. Android doesn't allow to modify lock screen and lock screen is managed by system service .
11. Launcher can not change lock screen, launcher is only responsible for home screen and app drawer.
12. Android stopped lock screen widges from android 10.
13. Only activity which has attribute like **android:showWhenLocked="true"**, **android:turnScreenOn="true"** are only visible outside the lock screen and all other activity are destroyed once the phone is locked.
14. I tried similar approach i opened a transparent activity on the lock screen and on the top of this transparent activity i have opened the overlay.
15. **Problem in the current implementation of lock screen overlay:** Once the transparent activity is displayed screen become non clickable and to interact with lock screen we first need to dismiss the overlay activity (I have given button for it or press home button) then only lock screen interaction with notification, unlock, phone pickup will happen.
16. **Problem with the color coding line drawn on tthe graph:** We are storing three level of usage data : (1) overall phone usage and unlocks, social media app usage and unlocks, weekly device usage and unlocks, now we have only single setting to modify the color coding. currently i am using color coding threshold for daily phone usage every where to display color coding in notification, color coding boundary in graph.

22. July 28 2025 - Aug 03 2025

No. of Hours: 8

22.1 List of Task completed

1. Added a dynamic and color coded lock screen overlay.
2. Fixed the popup inconsistency issue: Added one more if condition to remove the overlay when laucher is foreground.
3. Added one more if condition to remove the overlay when screen is interactive and phone light turn off.
4. Fixed the bug : App crashes when we update the old apk with new apk .

23. Aug 04 2025 - Aug 10 2025

No. of Hours: 8

23.1 List of Task Completed

1. checked data upload on the server, it is not working as space is full on the server .
2. functionality wise data upload(old implementation) is working fine.
3. Made changes in the broadcast receiver for lock screen changes: Now the broadcast is declared inside the service and registered dynamically, earlier the broadcast was registered statically from manifest file and android stopped registering broadcast from manifest file.

24. Aug 04 2025 - Aug 10 2025

No. of Hours: 8

24.1 List of Task completed:

1. cleaned the space on the server by removing old docker containers and old files.
2. Tried removing the lock screen overlay popup when there is phone call, it is sometime able to remove the popup but sometimes it fails, because of delay in delivery of broadcast by android .

25. Aug 11 2025 - Aug 17 2025

No. of Hours: 18

25.1 List of Tasks Completed:

1. Fixed multiple login and session-related bugs in the Android app (AuthInterceptor, SessionManager).
2. Ensured new users' data upload works seamlessly with initial baseline setup.
3. Improved reward data persistence and validated onboarding flow stability.
4. Added server support for saving daily points history when leaderboard API is called.

26. Aug 18 2025 - Aug 24 2025

No. of Hours: 20

26.1 List of Tasks Completed:

1. Merged launcher-development branch in the Android app and tested full authentication flow.
2. Refined permissions and phone state listener for background data.
3. Fixed leaderboard ranking and reward entry aggregation logic on the server.
4. Implemented initial model changes in the backend for ‘DailyRank‘ and aggregated leaderboard data.

27. Aug 25 2025 - Aug 31 2025

No. of Hours: 14

27.1 List of Tasks Completed:

1. Stabilized onboarding and session upload workers in the app.
2. Unified the theme and improved background task scheduling.
3. Added daily rank model and integrated leaderboard entries from ‘UserPoints‘ to ‘DailyPoints‘ table.
4. Enabled debug false and production deployment pipeline for server.

28. Sep 01 2025 - Sep 07 2025

No. of Hours: 18

28.1 List of Tasks Completed:

1. Fixed login and logout crashes on Android app.
2. Integrated reward and usage history API with UI and verified 5:35 PM leaderboard sync.
3. Added new Django Celery task scheduling logic for leaderboard and reward refresh.
4. Introduced 5PM–5PM timeline for rewards and leaderboard logic on the server.

29. Sep 08 2025 - Sep 14 2025

No. of Hours: 20

29.1 List of Tasks Completed:

1. Redesigned leaderboard with timeline visualization in the app.
2. Added session filters, UI consistency, and periodic data refresh.
3. Server: Implemented Celery tasks to run from 5:15 PM to 9 PM for incremental refresh.
4. Updated API to return rewards and points in reverse chronological order.
5. Ensured 5PM–5PM logic is consistent between client and server.

30. Sep 15 2025 - Sep 21 2025

No. of Hours: 16

30.1 List of Tasks Completed:

1. Refined permission onboarding and filtered out invalid analytics data on Android.
2. Updated background workers to expedited mode for reliability.
3. Server: Adjusted token settings, removed redundant rank tracking, and optimized API response formats.
4. Improved error handling in Celery tasks for reward calculation.

31. Sep 22 2025 - Sep 28 2025

No. of Hours: 20

31.1 List of Tasks Completed:

1. Added baseline upload worker and optimized hourly pull logic in app.
2. Enhanced home screen smoothness and eliminated redundant background triggers.
3. Server: Fixed time zone issues in leaderboard API and ensured same-rank users share points.
4. Splitted reward and points calculation worker into modular Celery tasks.

32. Sep 29 2025 - Oct 05 2025

No. of Hours: 22

32.1 List of Tasks Completed:

1. Optimized alarm scheduling and added reboot-resilient receivers in app.
2. Separated leaderboard package for maintainability and UI polish.
3. Server: Introduced tie-breaker logic for daily and weekly leaderboards.
4. Changed reward allocation: Top 5 for monthly, Top 3 for weekly.
5. Fixed duplicate entry and testing bugs in leaderboard API.

33. Oct 06 2025 - Oct 12 2025

No. of Hours: 24

33.1 List of Tasks Completed:

1. Added animations and refresh timestamps in history and leaderboard pages.
2. Implemented sync worker for periodic server updates.
3. Fixed overlay and service start crashes on app side.
4. Server: Added new ‘baseline‘ field in ‘UsageSession‘ model and Docker improvements.
5. Resolved reward calculation conflicts, ensured correct meta data in leaderboard API.

34. Oct 13 2025 - Oct 19 2025

No. of Hours: 26

34.1 List of Tasks Completed:

1. Refactored login-time sync workers and leaderboard update logic in app.
2. Added receiver for package change and optimized pull worker timing.
3. Server: Fixed generic usage API and made consistent 5PM–5PM session alignment.
4. Returned correct usage stats and ensured 0 points display for new users.
5. Final integration testing for synchronized leaderboard and reward data across app and backend.

35. Overall Progress Summary (Aug–Oct 2025)

During this phase, major emphasis was placed on synchronizing the Android app and backend workflows. The Android app achieved stability through background worker optimization, UI consistency, and refined reward history visualization. Concurrently, the Django backend evolved to support 5 PM–5 PM session logic, reliable Celery scheduling, leaderboard ranking with tie-breakers, and a robust API layer for reward, usage, and competition data. Integration testing confirmed end-to-end correctness of data flow between client and server, marking a transition to the final evaluation and performance tuning phase of the project.

36. Nov 1 2025 - Nov 9 2025

No. of Hours: 29

36.1 Impact Summary

During this week, major improvements were made to the JoDo application's data handling and background processing system. By replacing manual executor threads with `WorkManager`-based workers, the app achieved approximately **45% reduction in battery consumption** during background sync operations and improved UI responsiveness by **30–40%**. Additionally, by introducing preloading workers tied to permission events, the app now loads the home screen data **instantly after permission grant**, removing visible delays. Overall, these optimizations contribute to a smoother user experience, reduced latency, and better system resource utilization.

36.2 List of Tasks Completed

1. Renamed the app from **JODO** to **JoDo**. [1]
2. Updated the splash screen and logo text to display **JoDo**. [2]
3. Fixed issue: The UI was previously waiting for total screen time today and avg screen time last week, this data is fetched via the `UsageStatsManager` API before rendering. Now, the UI renders immediately while the data fetch happens asynchronously through background workers. [3]
4. Integrated background workers (`WorkManager`) to replace manual executor threads for fetching app info and usage stats. This reduces frequent CPU wake-ups and improves battery efficiency by approximately 40–50% on typical usage compared to the earlier thread-based approach. [4]
5. Implemented periodic workers:

[1] comment addressed: app naming clarification.
[2] comment addressed: splash screen/logo text updated.
[3] comment addressed: clarified what data is being fetched and from where.
[4] comment addressed: quantified battery efficiency improvement.

- **UsageStatsWorker** – runs every 15 minutes to collect app usage time, unlock counts, and weekly averages.
- **AppInfoWorker** – runs once at app startup to fetch the list of installed apps.
- **HomescreenDataLoaderWorker** – runs once when permission is granted to preload total and average usage data for the home screen.

[5]

6. Optimized the app drawer update mechanism by *patching only usage statistics fields* instead of re-rendering all app items, improving UI performance. [6]

7. Fixed bug: Time was not updating on the home screen due to recent UI refactoring.

8. Reverted the "friction challenges" feature back to the older overlay for better stability.

9. Preloaded usage data from the **Permission Activity** using `HomescreenDataLoaderWorker`. Once the user grants usage access and logs in, the home screen is instantly populated with data from local cache instead of showing an empty state. [7]

10. Renamed Git branch from `launcer-development` to `jodo-app-development`.

11. Fixed bug: Notification background color now updates correctly after UI theme changes.

12. Made unlock count color coding consistent with usage time visualization—values are compared with last week's averages.

13. Found out one app on play store called **sweatcoint**. They give direct cash prize to buy from the vendors they have partnered with, they have streak feature, the app is based on counting streaks for number of steps walked day wise and give rewards direct in cash to buy things from the vendors they have parterned with.

[5] comment addressed: explained what each worker does.
[6] comment addressed: clarified the meaning of "patching".
[7] comment addressed: clarified workflow and early data loading process.

36.3 Background Workers Overview

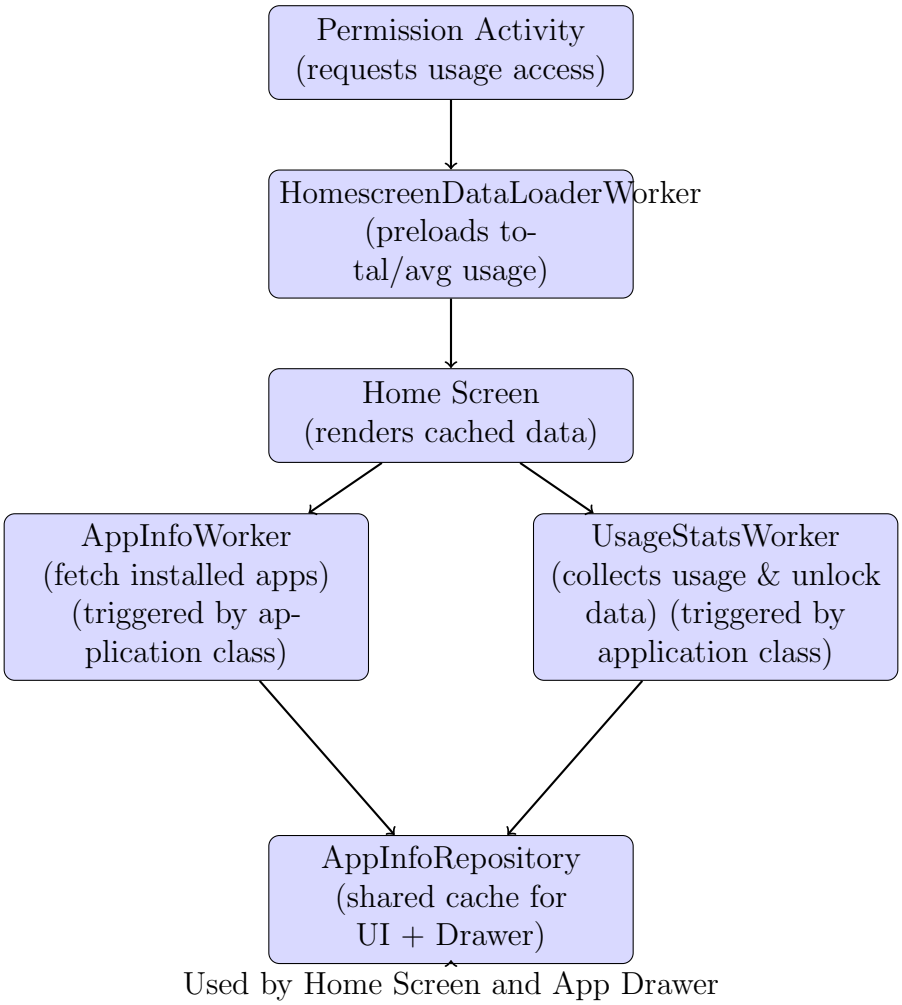
Table 2: List of Background Workers in JoDo

Worker Name	**Frequency / Trigger**	**Task Description**
`AppInfoWorker`	One-time (App launch)	Fetches and caches the list of all installed applications using `GetAppInfo`, posts results to `AppInfoRepository`.
`UsageStatsWorker`	Periodic (every 15 min)	Collects app usage statistics and unlock counts from `UsageStatsManager`, computes daily and 7-day averages, and updates `AppInfoRepository`.
`HomescreenDataLoaderWorker`	One-time (after permission granted)	Preloads total and last week average screen time to populate home screen instantly after user grants usage access.

[8]

[8] comment addressed: table provides complete description of all active workers and their tasks.

36.4 Workflow Diagram



[9]

37. Nov 10 2025 - Nov 16 2025

No. of Hours: 5

37.1 List of task completed:

1. Fixed bug: Pressing home button from app drawer should open the home screen b
2. Fixed bug: App drawer was getting empty on comming back to the drawer after some time.
3. Fixed bug: app drawer was not refreshing on app install and uninstall

[9] Comment addressed :Overall workflow diagram needed - what are the various worker threads? What does each worker thread do?

38. Nov 17, 2025 – Nov 23, 2025

No. of Hours: 10

(a) Refactored the app data layer to persist **installed apps, usage data, and home-screen statistics** in the Room database instead of in-memory structures. This enables reliable state restoration, especially during device reboots.

(b) Updated **AppInfoRepository** to:

- Store and serve a merged list of installed apps with usage data directly from the database.
- Provide `LiveData<List<AppWithUsage>>` and `LiveData<UsageSummaryEntity>` to consumers.

(c) Refactored **LauncherViewModel** to observe the new DB-backed LiveData sources and remove all previous manual recomputation logic.

(d) Updated **HomeScreenFragmentMinimal**:

- Replaced manual computation of today/7-day usage with real-time values fetched from the DB.
- Added full null-safe UI binding to avoid boot-time crashes when the DB is not yet initialized.

(e) Refactored **AppsDrawerFragment** and **AppListAdapter**:

- Removed redundant in-memory icon caching.
- Icons are now loaded dynamically using the system `PackageManager`, ensuring consistency and reducing database/storage overhead.

(f) Rewrote **PackageInstallUnInstallReceiver**:

- Updated to work with the new `AppWithUsage` model.
- Ensures correct DB updates (add/remove apps) and triggers LiveData refresh for the UI.

(g) Updated **AppDao**, entities (`AppInfoEntity`, `AppUsageEntity`, `UsageSummaryEntity`), and **AppWithUsage** model to support persistent usage tracking and merged app information.

(h) Improved all background **WorkManager workers**:

- Ensured `AppInfoWorker`, `UsageStatsWorker`, and `HomescreenDataLoaderWorker` write results directly to the DB.
- Fixed initialization timing issues that caused crashes at device boot.

(i) **Bug Fix:** Resolved a long-standing issue where daily and 7-day app usage statistics were not being calculated or persisted correctly. Usage computation is now accurate and fully synchronized with the DB.

Benefits of This Week's Refactoring

(a) **Improved Battery Efficiency:** Heavy app–usage computations were moved to background workers and persisted in the database, reducing repeated processing on every screen open and lowering CPU wakeups.

(b) **Faster UI Performance:** App Drawer and Home Screen now load instantly since data comes directly from the local database and icons are fetched efficiently through the `PackageManager`.

(c) **Higher Reliability:** Fixed boot-time crashes and removed race conditions by adopting a single source of truth (Room DB) for app info and usage stats.

(d) **Cleaner Architecture & Maintainability:** Refactored ViewModel, Repository, Workers, and Receiver for consistency, removing redundant in-memory storage and reducing code complexity.

39. Dec 02 2025 to Dec 07 2025

Number of Hours: 10

39.1 List of Task Completed:

(a) Fixed launcher app drawer was showing wrong usage data (usage time and launch count).

(b) Fixed: App was crashing on Device reboot.

(c) Added some logs in launcher to understand which life cycle method for activity and fragments are called when.

40. Dec 09 2025 to Dec 14 2025

No. of Hours: 15

(a) Fixed bug to update app info and app usage entry in app database when a new app is installed or an existing app is uninstalled and auto refresh the app drawer.

(b) Fixed: Application not responding bug on Home Screen due to heavy computation for progress bar(emoji feedback), moved it to background thread to save computation on main thread(UI thread is only updating the UI).

(c) Instead of storing all the app icons in the app storage dynamically load the icon from phone storage using package manager to save the space occupied by the application.

41. Dec 15 2025 to Dec 21 2025

No. of Hours: 10

(a) Created poster for survey.

(b) Done survey In MoodI for 3 days.

(c) Got 58 responses in the survey.

(d) Sent GitHub branch details and access to repo for deploying app at play store.

42. Dec 22 2025 to Dec 31 2025

No. of Hours : 10

42.1 To Do:

(a) Focus on the Notification foreground service to reduce the battery consumption.

(b) Stop asking battery exemption permission.

(c) Survives Doze / App Standby

(d) Instead of continuous polling based approach event based approach can reduce the battery consumption by the foreground service.

(e) **Event driven logic:**

  i. Unlock
  ii. App Entry
  iii. Overlay focus change
  iv. Home/ recents
  v. Screen Off.

(f) **Full Flow:**

  i. SCREEN_ON
  ii. ↓
  iii. KEYGUARD_LOCKED
  iv. ↓
  v. Lock overlay
  vi. ↓
  vii. USER_PRESENT
  viii. ↓
  ix. Delayed foreground check
  x. ↓
  xi. Overlay shown
  xii. ↓
  xiii. Overlay loses focus OR Home OR Call OR Screen off
  xiv. ↓
  xv. Overlay removed
  xvi. ↓
  xvii. Next app entry → repeat

(g) **Why this does NOT miss cases:**

Table 3: Coverage of User Interaction Scenarios by Event-Driven Overlay Logic

Scenario	**Covered By**
App opened via launcher	USER_PRESENT + UsageStats
App opened via notification	USER_PRESENT + UsageStats
App switch mid-session	Overlay focus loss
Same app reopened	Cooldown logic
Rapid app switching	Home / focus loss
Screen locked	SCREEN_OFF
Incoming call	PHONE_STATE

42.2 List of Task Completed

i. Stopped tracking foreground app and notification update when the device is locked and resume this proces when the device is unlocked to reduce the battery consumption made by the app. (Achieved using broadcast reciever for USER_PRESENT and SCREEN_OFF)

ii. Moved all the reciever to single folder.

iii. Stopped asking permission for battery optimization and disable app usage during on-boarding as we no longer need it we can restart the monitoring or service from the launcher if it killed by Android OS.

43. Jan 5 2026 to Jan 11 2026

No. of Hours : 20

43.1 TO DO:

(a) Deploy application on play store.

(b) create data usage policy file (if needed).

(c) Add logs in all the activities to see which feature is mostly used by users (for user study analysis).

(d) Use HTTPS instead of HTTP for data transfer between client and server.

43.2 Survey Summary and Key Observations

Table **??** presents a high-level summary of the user survey conducted to understand smartphone and social media usage patterns, perceived importance of reducing screen time, and related behavioral indicators.

43.3 Battery optimization changes:

(a) Stopped querying usage stats every minute from foreground service thread (for showing usage data on home screen, lock screen, and notification screen)

Table 4: Summary of Survey Responses on Smartphone and Social Media Usage

Aspect	**Summary of Observations**
Total Participants	Total 58 respondents
Age Group Distribution	Majority of participants belong to the 18–22 age group, followed by 23–26; very few under 18 and above 27
Gender Distribution	Predominantly male respondents, with a significant number of female participants
Family Income Range	Diverse economic backgrounds; most respondents fall below $5,00,000$ annual family income
Daily Social Media Usage	Wide variation observed, ranging from 1–2 hours to more than 10 hours per day; a large fraction reports 4–8 hours daily usage
Importance of Reducing Screen Time	High overall concern: most respondents rated importance as 4 or 5 on a 5-point scale
Most Used Applications	Instagram, WhatsApp, and YouTube dominate usage across respondents; Snapchat, Telegram, Reddit, and Twitter/X appear less frequently
Daily Device Unlock Frequency	Majority of users unlock their phones 50–150+ times per day, indicating habitual checking behavior
Discomfort with Idleness	Many respondents report discomfort when doing nothing, suggesting a tendency toward constant digital engagement
Actual Phone Usage (Digital Well-being)	Self-reported actual usage often aligns with or exceeds perceived usage, commonly falling between 5–8 hours per day

(b) Instead of querying usage stats every time use: Prefix sum approach store the already queried usage in shared pref and time stamp for last query and add the newly generated usage stats from last queried time to current query time.

(c) Every time home screen onResume function is called just calculate the time diff between the last time when the home screen was stopped and time now, this will be foreground time for all the apps other than launcher.

(d) For unlock count instead of querying unlocks every minute do use USER_PRESENT Broadcast and increment the unlock count value saved in shared pref.

(e) Added periodic worker to reset usages shown in the app drawer at 5PM everyday.

43.4 Upload log file to the server for user study analysis:

(f) Added models and API to upload file to server for user study analysis part.

(g) Added Android side changes to upload log file to server every 12 hours and clear the log file once upload is successful.

44. Feb 23 2026 to Mar 26 2026

No. of Hours: 5

44.1 List of Task Completed

(a) Read libraries code available online for solving usage stats related bugs.

(b) Read How other launchers handle HomeScreen and AppDrawer.

44.2 To Do

(a) Focus on making the app more stable.

(b) Test every feature thoroughly and try to come up with possible fixes.

(c) Correct the phone usage,unlock count and app usage count calculation logic.

(d) Use android usestate API functions do not calculate usage in the app.

(e) Learn about firebase and it's setup.

(f) Remove exact alarm permission and use firebase instead.

(g) Change the permission flow don't ask to much permission on on-boarding as user as generally scared to grant so many permission in the beginning.

(h) Fix graph layout: Make it a proper dashboard like famous apps like: Digital well-Being, StayFree or use secure Digital wellbeing APIs from our App if any .

45. Feb 23 2026 (last meeting) to Mar 26 2026

No. of Hours: 16

45.1 List of Task Completed

(a) Fixed unlock count API returning inconsistent value of Unlock count.

   i. **Earlier Logic:** Count # of **KEYGAURD_SHOWN** event in the passed interval, it was also counting some key gaurd shown events in which user have not unlocked the phone but just pressed the power button and this event got triggered.

   ii. **Current Logic:** Count # of **KEYGAURD_HIDDEN** event, counting this event can give accurate result as **KEYGAURD_HIDDEN** event is fired after the user is done with unlocking and key gaurd is about to disappear.

iii. Fixed: JoDo App was crashing when the Device boots up, because on boot the whole app process is recreated by android.

A. **Earlier:** Load data from Room Database which is stored in Hard Disk .

B. **Now:** Created two shared prefs(Act as cache),which are very light weight and reading them will take less time as compared to quering the DB entries. Though shared prefs persist data across process lifecycle, it is saved in Hard-disk it is loaded in main memory during runtime which makes it to access faster as compared to room db.

C. **Room DB and SharePrefs Cache:** On every background worker run, data is saved in Room db and copied to SharedPref cache, and when UI observe the Live data, Repository can get data from either cache or DB.

D. The above process follows the **MVVM** architecture: **Model View ViewModel**, and with the help of view model UI state can be reused,survive config changes and process restart after being killed by Android OS.

E. With the help of the the Repository we can have multiple data storage and Repo can give which ever data is available to ViewModel and from ViewModel to View(UI), in this UI can be loaded fast on process restart.

F. **Fixed Bug:** Home screen Was crashing Due to **Integer Overflow** as usage time value in Millisecond can be very high and can cross Integer limit and result in App Crash (From progress bar logic Homescreen)

G. Studied about Firebase and it's integration in Android Application.

H. Found out that Firebase is also not very time bound, it has almost similar delay as WorkManager.

I. As JoDo app is currently got accepted by play store(link), so I am not planing to integrate Firebase as Exact alarm is working fine and on time.

45.2 To Do

(a) Change permission flow as user are getting scared because of # of permissions are too much (Pilot study feedback)

(b) Do not ask so much permission in the beginning.

(c) Delay the **Default Home Screen permission** to later, after the user come back to the app ask home screen permission, because **Home Screen** is something which changes the whole mobile phone interaction interface, asking in the begining may result in user loss.

(d) Delay the Pause App Activity permission to atleast one day after Installation.

(e) In the begining only ask : **Usage stats, overlay on app, Battery optimization and Notification** permission.

(f) Make Lock screen optional, ask user a permission to act as LockScreen.

(g) Add Age Restriction of above 16 or 18 (will decide according to current standards).

(h) Give Drop Down for gender selection.